

Fiche de procédure du cours openclassrooms sur Symfony 7

<https://symfonycasts.com/courses#tracks>

Partie 1 : Créez un projet Symfony :

Définitions :

- Symfony CLI : Outil en ligne de commande facilitant la création et la gestion de projets Symfony.
- Doctrine ORM : Couche d'abstraction qui permet de manipuler la base de données à travers des objets PHP.
- Twig : Moteur de template pour PHP offrant une syntaxe expressive pour générer du HTML dynamiquement.

Fiche résumé du cours :

En résumé

- Les frameworks vous aident en vous évitant d'avoir à réinventer la roue à chaque application.
- Symfony est un framework organisé en composants et bundles pour répondre à la plupart de vos besoins de développement.
- Pour installer des dépendances externes comme Symfony, utilisez Composer.
- Enfin, pour vous faciliter la tâche, utilisez le Symfony CLI pour installer et gérer vos applications Symfony plus facilement.

La requête

Tout d'abord, regardez la première ligne de la requête. Dans l'ordre, les trois éléments que nous avons sont : une méthode HTTP (ou *verbe HTTP*, "GET"), une ressource (le chemin auquel on tente d'accéder, "/request"), et la version du protocole HTTP qui a été utilisée (ici "HTTP/2"). Les deux premiers éléments vont nous être utiles très rapidement dans les prochains chapitres.

Premièrement, la méthode HTTP. Il y en a plusieurs possibles, chacune ayant une signification spécifique. Elles permettent au client d'indiquer au serveur le type d'action qu'il souhaite effectuer. Ici, "GET" indique que la requête a pour but de récupérer des informations. Ici encore, pas besoin de les connaître toutes, les suivantes suffiront :

Méthode	Signification
GET	Récupérer les informations disponibles à cette adresse
POST	Envoyer des données en laissant le serveur se charger de les traiter
PUT	Remplacer toutes les données à l'adresse indiquée
DELETE	Effacer toutes les données à l'adresse indiquée

La réponse

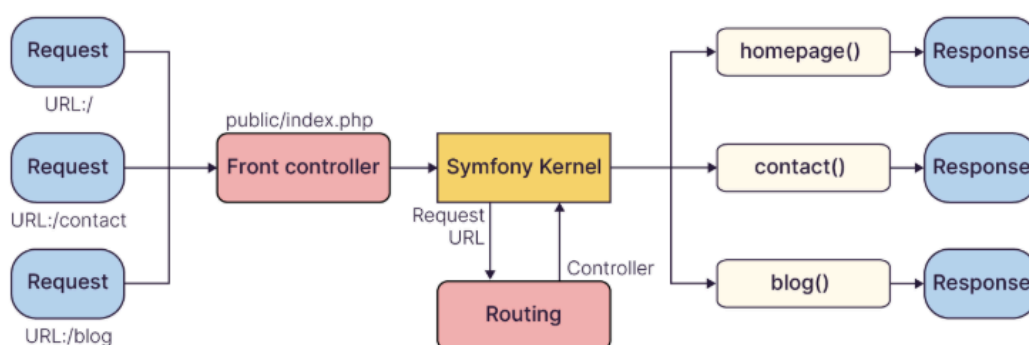
La réponse a la même structure que la requête, mais le contenu est différent. Cette fois, nous avons la version du protocole HTTP utilisée en premier. Ensuite, viennent deux informations primordiales : un code de réponse, suivi de sa version textuelle. Ce code nous offre une information instantanée sur l'état de notre réponse.



Et je suis censé connaître tous les codes ?

Pas du tout : il en existe un grand nombre. Vous connaîtrez la plupart au fil du temps, mais pour commencer, il suffit de connaître les principaux et de savoir qu'ils sont répartis en catégories :

Catégorie	Signification
1xx	Informations, généralement pour des réponses temporaires, ou suivies d'autres réponses
2xx	Succès, tout s'est bien passé et la réponse comporte l'information demandée
3xx	Redirection, lorsque la ressource a changé d'adresse
4xx	Erreur dans la requête, lorsque le format est mauvais, ou que la ressource demandée n'existe pas, par exemple
5xx	Erreur serveur, si l'application a planté, par exemple



En résumé

- Dans le protocole HTTP, un Client envoie une Requête à un Serveur, qui lui renvoie une Réponse
- Une requête se définit entre autres par une méthode et une ressource.
- Une réponse se définit entre autres par un code et un contenu.
- Le but de Symfony est de nous aider à traiter des requêtes et renvoyer ces réponses.
- Le Kernel de Symfony gère la configuration de notre application et exécute le traitement de la requête.

Partie 2 : Affichez des pages dynamiques :

Créez un schéma en base de données :

DBAL = DataBase Abstraction Layer

ORM = Object Relational Mapper

DSN = Database Source Name

MPD = Modèle physique de données

- La commande `make:entity` permet de créer rapidement des entités en étant sur du mapping généré.
- Les propriétés générées le sont avec leurs accesseurs et mutateurs (méthodes `getXxx` et `setXxx`).
- Une propriété `id` est systématiquement générée sans avoir besoin de la demander à la commande, car un `id` est nécessaire dans chaque entité pour utiliser Doctrine.
- Cet `id` n'a pas de méthode `setId` correspondante, car nous devons laisser Doctrine le gérer pour nous.

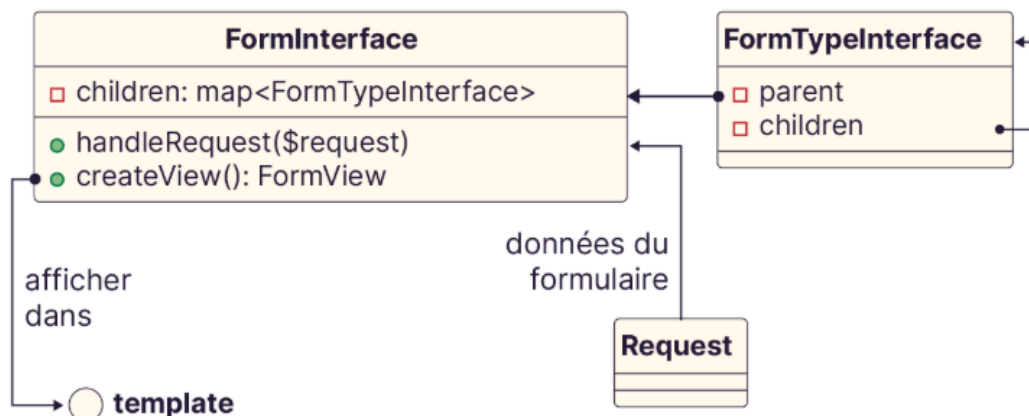
- **ManyToOne** : Ici, un objet A peut être lié à un objet B uniquement, mais chaque objet B peut être lié à plusieurs objets A. Exemple : un Book peut avoir un seul Editor, mais chaque Editor peut contenir plusieurs Books. Dans la base de données, avoir une colonne dans la table des éditeurs qui contiendrait tous les identifiants des livres liés serait impossible à maintenir. Nous aurons donc au contraire une colonne dans la table des livres qui référencera l'éditeur qui a publié ce livre. Et si plusieurs livres font référence au même éditeur, pas de problème. Le côté de cette relation sur lequel est définie la relation (la table des produits) est appelé "propriétaire". Le côté propriétaire sera toujours le côté "Many" d'une relation de ce type. Ce terme sera important par la suite.
- **OneToMany** : Il s'agit de la même relation que précédemment, mais observée depuis le côté non propriétaire. Cela signifie que notre relation est "bidirectionnelle" : en effet, on cherche à définir une relation du côté "One" (côté Editor, dans notre exemple), mais sa représentation en base de données sera du côté "Many". Nous sommes donc obligés de la définir des deux côtés à la fois.
- **ManyToMany** : Ici, un objet A peut référencer plusieurs objets B, mais un objet B peut aussi référencer plusieurs objets A. Par exemple, un même objet Author pourrait référencer plusieurs Books, et un Book pourrait faire référence à plusieurs objets Author suivant le nombre d'auteurs qui ont pris part à l'écriture. Ici, nous ne pouvons toujours pas avoir en base une colonne dans Author qui contiendrait tous les identifiants de Books, ni de colonne dans Book pour tous les Authors. Doctrine va donc créer pour nous une *table de jointure*. Cette table s'appellera `book_author` (ou `author_book`, peu importe) et ne contiendra que deux colonnes : `author_id` et `book_id`. Du coup, nous pouvons avoir plusieurs fois le même identifiant d'auteur, et plusieurs fois le même identifiant de livre. Seule la combinaison des deux doit être unique.
- **OneToOne** : C'est la relation la plus simple, un objet A ne peut être lié qu'à un objet B, et un objet B ne peut être lié qu'à un objet A. Vous ne pouvez avoir qu'une seule carte d'identité, et votre carte d'identité ne peut appartenir qu'à vous, par exemple. Une simple colonne avec une clé étrangère dans l'une ou l'autre des tables (ou les deux) suffira amplement.

En résumé

- Doctrine est l'outil qui nous sert à gérer les bases de données relationnelles avec Symfony.
- La configuration de doctrine se fait par un DSN (Database Source Name) dans le fichier `.env`
- Une entité est un mapping que Doctrine utilise pour faire le lien entre nos classes PHP et la base de données.
- Nos classes et nos entités peuvent avoir des relations complexes.
- MakerBundle permet de créer des entités de manière interactive, rapide et sûre.
- Les Migrations sont le moyen privilégié d'appliquer des modifications structurelles à notre base de données.

Récoltez des données à sauvegarder :

Le composant Form va vous permettre de créer, gérer et afficher des formulaires très facilement.



Normalement, votre classe devrait ressembler à peu près à ça :

```
1 <?php
2 // ...
3 use Symfony\Component\Form\Extension\Core\Type\DateType;
4 use Symfony\Component\Form\Extension\Core\Type\TextType;
5 // ...
6 public function buildForm(FormBuilderInterface $builder, array $options): void
7 {
8     $builder
9         ->add('name', TextType::class)
10        ->add('dateOfBirth', DateType::class, [
11            'input' => 'datetime_immutable',
12            'widget' => 'single_text',
13        ])
14        ->add('dateOfDeath', DateType::class, [
15            'input' => 'datetime_immutable',
16            'widget' => 'single_text',
17            'required' => false,
18        ])
19        ->add('nationality', TextType::class, [
20            'required' => false,
21        ])
22        ->add('books', EntityType::class, [
23            'class' => Book::class,
24            'choice_label' => 'id',
25            'multiple' => true,
26            'required' => false,
27        ])
28    ;
29 }
```

Et voilà ! Votre premier `FormType` est prêt à être utilisé.

Passons maintenant au template. Pour afficher un formulaire dans un template, on passera notre variable `form` à plusieurs fonctions :

- `form_start(form)` génère l'affichage de la balise d'ouverture de notre formulaire ;
- `form_widget(form)` génère l'affichage des champs de notre formulaire, avec leurs labels et erreurs éventuelles ;
- alternativement, appelez `form_row(form.[nom_d_un_champ])` pour afficher un seul champ avec son label et son erreur éventuelle ;
- `form_end(form)` génère l'affichage de la balise de fin de notre formulaire, ainsi que l'affichage de tous les champs que vous auriez oublié d'afficher.

En résumé

- Le composant Form sert à créer, gérer et afficher nos formulaires.
- Nos formulaires sont représentés sous forme de classes étendant `AbstractType` .
- Ces objets peuvent être utilisés dans nos controllers pour récupérer les données envoyées par l'utilisateur dans la requête et les hydrater dans nos entités.
- Twig dispose de fonctions pour faciliter l'affichage de formulaires.

Partie 3 : Approfondissez votre usage de Symfony

Définissez les utilisateurs :

Détaillons-les rapidement.

Pour `UserInterface` :

- Nous avons `getUserIdentifieur` qui permet de retourner la chaîne de caractères qui identifie notre utilisateur quand il se connecte, qui est aussi appelé `login` sur plein de sites internet. Ce peut être une chaîne arbitraire appelée ici `username`, ou son adresse e-mail, ou ce que vous voulez d'autre. Rappelez-vous juste que c'est ce que l'utilisateur doit entrer dans un formulaire de connexion, donc le plus simple est généralement le mieux.
- `getRoles` permet de retourner un array de chaînes de caractères appelées roles, et qui permet d'aider à déterminer les droits de l'utilisateur. Nous reviendrons dessus au chapitre 3 de cette partie.
- `eraseCredentials` est une fonction dont nous pourrions nous servir pour éviter de garder en mémoire le mot de passe de l'utilisateur avant hachage, mais ce n'est pas obligatoire.

Du côté de `PasswordAuthenticatedUserInterface` c'est plus simple, nous avons uniquement `getPassword`, qui permet de récupérer le mot de passe haché. Nous reviendrons sur cette notion au chapitre suivant.

Notez que comme nous utilisons déjà Doctrine, le plus simple pour nous sera de sauvegarder nos utilisateurs en base de données. Fort heureusement, rien ne nous empêche d'implémenter ces interfaces sur une entité, c'est même tout à fait classique. Et c'est donc ce que nous allons faire.

- MakerBundle vous propose une commande `make:user` spécifiquement pour créer une classe qui implémente `UserInterface`.
- Grâce à cette commande, vous pouvez aussi choisir d'implémenter `PasswordAuthenticatedUserInterface` ou non.
- Vous pouvez aussi choisir de faire de votre classe une entité ou non.
- La commande gère pour vous les changements à apporter à la configuration de sécurité dans `config/packages/security.yaml`.

En résumé

- La sécurité dans Symfony fonctionne en deux étapes : Authentification et Autorisation.
- Pour représenter l'utilisateur qui déclenche une requête, Symfony a besoin d'un objet qui implémente `UserInterface`.
- Cet objet utilisateur est chargé durant l'authentification au moyen d'un `UserProvider`.
- Symfony embarque nativement plusieurs types de `UserProviders`, dont le type entité qui cherche les utilisateurs en base de données.

Mettez en place l'authentification :

- Vous pouvez créer rapidement une ébauche de formulaire d'enregistrement grâce à la commande `make:registration-form` .
 - La commande ajoute un attribut pour s'assurer de l'unicité de chaque utilisateur en base de données.
 - La commande crée aussi un FormType adapté qui prend en paramètre le mot de passe de l'utilisateur en clair.
 - Le controller généré fait ensuite appel à un objet pour hacher ce mot de passe.
-
- On peut demander des contrôles d'accès grâce aux nombreuses versions de la méthode `isGranted` .
 - Ensuite, Symfony appelle des Voters, qui peuvent s'abstenir, accorder l'accès ou le refuser.
 - Le comportement par défaut vérifie des rôles.
 - On peut créer nos propres Voters pour implémenter des restrictions spécifiques.
 - La section `access_control` du fichier `security.yaml` permet de restreindre des sections

Nom de l'événement	Classe	Quand ?
kernel.request	RequestEvent	Quand le Kernel commence à traiter la requête. Le routing est un Listener qui a lieu pendant cet événement. Permet de renvoyer une réponse anticipée en stoppant la propagation en cas de besoin.
kernel.controller	ControllerEvent	Quand le routing a identifié la route et le controller associé.
kernel.controller_arguments	ControllerArgumentsEvents	Quand le routing a vérifié que le controller existe et identifié les arguments qu'il demande.
kernel.view	ViewEvent	Quand le controller a été exécuté, seulement s'il ne retourne pas d'objet Response.
kernel.response	ResponseEvent	Quand le controller a été exécuté et qu'on a un objet Response, après <code>kernel.view</code> (s'il a été dispatché), pour vous permettre de modifier l'objet Response.
kernel.finish_request	FinishRequestEvent	Après <code>kernel.response</code> , pour vous permettre d'inspecter l'objet Request une dernière fois.

kernel.terminate

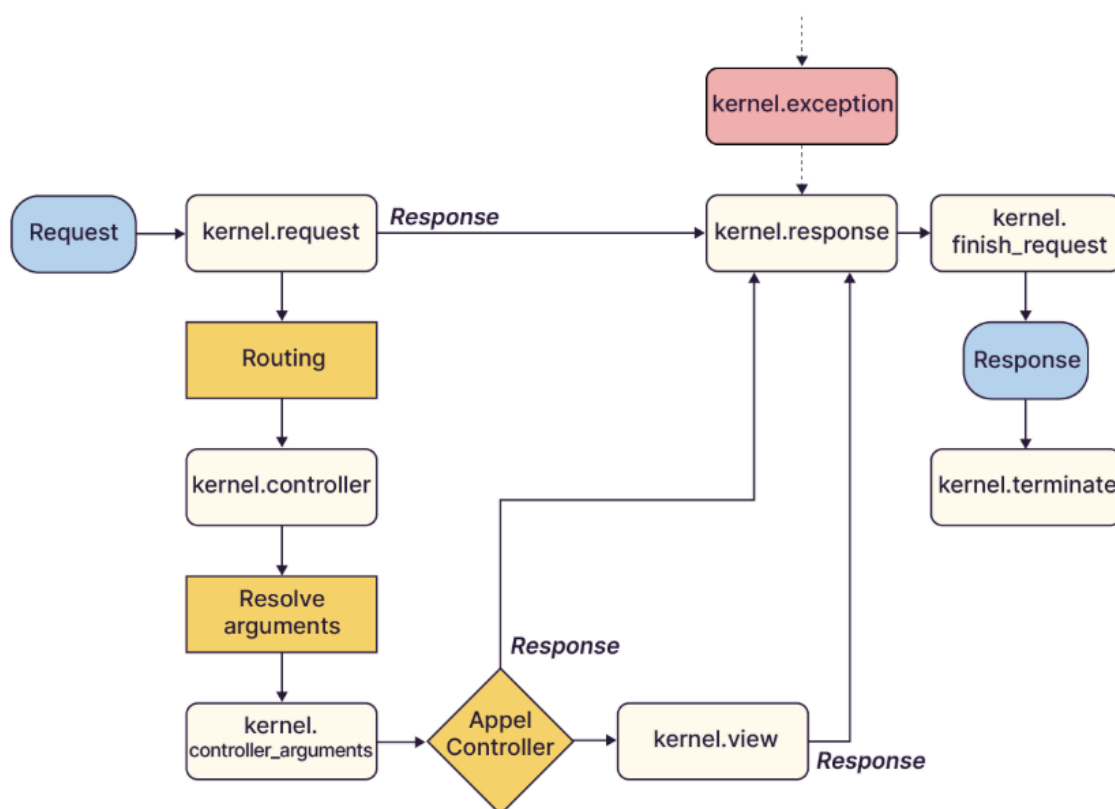
TerminateEvent

Après que la réponse a été renvoyée au client, pour vous permettre de faire des traitements longs sans pénaliser l'utilisateur.

kernel.exception

ExceptionEvent

N'importe quand pendant que le Kernel traite la requête, si une exception est levée mais que vous ne l'attrapez pas. On reprend ensuite le flux normal depuis `kernel.response` pour renvoyer notre exception sous forme de message d'erreur.



En résumé

- L'EventDispatcher permet de dispatcher et d'écouter des événements pour découpler notre code.
 - De nombreux composants Symfony dispatchent leurs propres événements, dont le Kernel.
 - Les événements du Kernel permettent de se brancher sur le fonctionnement interne de Symfony pour en personnaliser l'exécution.
 - Pour écouter un événement, on peut créer des EventListeners ou des EventsSubscribers.
-
- Symfony est un framework libre et open source qui n'appartient à aucune entreprise.
 - Symfony suit la norme SemVer pour ses numéros de versions.
 - Un calendrier de release très précis a été mis en place grâce à cette norme.
 - La Backward Compatibility Promise de Symfony assure que les changements apportés à Symfony seront toujours rétrocompatibles au sein d'une même version majeure.
 - Les dépréciations vous aident à identifier à l'avance les changements non rétrocompatibles.
 - N'importe qui peut proposer un changement de code ou de documentation, mais seule la Core Team peut valider ces changements.

TWIG

`{# commenter quelque chose #}`
`{% faire quelque chose %}`
`{{ afficher quelque chose }}`

```
{% extends 'base.html.twig' %}
```

Cette ligne indique que notre template en étend un autre. Vous n'avez pas manqué de remarquer que beaucoup de choses manquent dans ce fichier pour en faire une page HTML complète